LAB 09 — Bias, Variance and Imbalanced Data

Overfitting, ROC/AUC, Resampling, SMOTE and Noisy Data on Real Medical Data

Course	Machine Learning — BS Computer Science, 5th Semester
Duration	3 hours (single session)
Dataset	Stroke Prediction Dataset (Kaggle) — 5,110 patients, 10 health/lifestyle features, target: stroke
Dataset link	https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset (download healthcare-dataset-stroke-data.csv)
Builds on	Labs 3-8 (train/test split, scaling, Decision Tree, Random Forest, Logistic Regression, cross-validation, metrics)
New library	imbalanced-learn (for SMOTE and resampling). Install once: pip install imbalanced-learn
Deliverable	One completed notebook (see Submission Requirements)

Learning Objectives

- Explain why accuracy is misleading when one class is rare, and choose better metrics.
- Recognise underfitting (high bias) and overfitting (high variance) from validation curves, learning curves and a direct variance measurement.
- Read and compare ROC and Precision-Recall curves, explain what AUC does and does not tell you, and choose a decision threshold.
- Apply random undersampling, random oversampling and SMOTE correctly (on the training data only) and judge what they really change.
- Show how label noise and feature noise hurt different models, and why SMOTE can make noisy data worse.
- Select and justify a final approach for a medical screening problem.

Before You Start

- You already know the models, scaling, splitting and cross-validation from Labs 3-8. This lab does not re-teach them.
- This is a thinking lab. Almost every task ends with a question that asks you to explain what your own results mean. A table or plot without your written interpretation earns no marks.
- Give objective + question style: you decide which pandas / scikit-learn tools to use. Hints only point to the right tool.
- Use SEED = 42 everywhere (random_state) so your results can be compared with your classmates' and checked by the instructor.
- Golden rule of the whole lab: anything that learns from data (scaler, imputer, resampler, threshold) is learned from the TRAINING data only.

Part 0 — Setup and the Data (10 minutes)

A hospital wants to flag patients at risk of stroke from routine information. The target column is stroke (1 = had a stroke, 0 = did not).

1. Install imbalanced-learn (pip install imbalanced-learn) and import everything you need.
2. Load the CSV. Report the shape, the columns with missing values, the value counts of gender and the number of patients with stroke = 1 and stroke = 0 (count and percentage).
3. Clean the data: remove the identifier column; remove any gender category that has too few rows to learn or test; apply categorical encoding to the text columns (pd.get_dummies, as in the labs).
4. Split 80/20 with stratify=y and random_state=SEED. Fill the missing numeric values with a median learned from the TRAIN set only. Create a scaled copy (StandardScaler fitted on train) to use for Logistic Regression.
5. Report how many stroke cases are in the training set and in the test set.

Question

(a) In 2-3 lines: how imbalanced is the data? Why did we use stratify=y, and what could go wrong with only a few stroke cases in the test set?

Part 1 — The Accuracy Trap (15 minutes)

Objective

Show that a model can have excellent accuracy and still be useless.

1. Build a baseline that always predicts the majority class (DummyClassifier, strategy='most_frequent'). Report accuracy.
2. Train a default Logistic Regression on the scaled training data. Report Accuracy, Precision, Recall, F1, ROC AUC and PR AUC (average precision) on the test set, and plot or print its confusion matrix.
3. Write a small helper function that takes a model and train/test data and returns these six metrics - you will reuse it all lab.

(a) Compare the accuracy of the baseline and of Logistic Regression. What do you conclude?

(b) Look at the confusion matrix and the recall. In the hospital's situation, what does a false negative mean, and which numbers should we trust instead of accuracy?

Part 2 — Underfitting, Overfitting, Bias and Variance (40 minutes)

A Decision Tree is a good model to study this because one parameter, max_depth, moves it smoothly from too simple to too complex. From now on use ROC AUC (not accuracy) as the score for cross-validation.

Task 2.1 — Validation curve

1. For max_depth = 1, 2, ..., 15 and None, record the train ROC AUC and the 5-fold stratified cross-validation ROC AUC (train data only).
2. Plot both curves on one chart (labelled axes, title, legend). Report the best depth by CV AUC.

Task 2.2 — Learning curves

1. Use sklearn.model_selection.learning_curve (scoring='roc_auc', train_sizes from 20% to 100%) for a tree with max_depth=1 and a tree with max_depth=None. Plot train and CV AUC against the number of training rows, side by side.

Task 2.3 — Measure variance directly

Variance means: if the training data changes a little, how much do the model's predictions change? Measure it.

1. For each of max_depth = 1, 3 and None: draw 40 bootstrap samples of the TRAIN rows (sample with replacement, same size), fit a tree on each, and collect each tree's predicted stroke probability for every test patient.
2. For every test patient, compute the variance of its 40 predicted probabilities; then average over patients. Also report the mean test ROC AUC of the 40 trees and the ROC AUC of the averaged prediction (mean of the 40 probability vectors).
3. Show the three depths in one table.

Hints

- rng.choice(len(X_train), len(X_train), replace=True) gives bootstrap row positions; use X_train.iloc[idx].
- Collect predictions in an array of shape (40, number of test patients); variance across models is probs.var(axis=0).
- learning_curve returns (train_sizes, train_scores, valid_scores) - average the scores over the folds with .mean(axis=1).

(a) From the validation curve: which depths underfit, which overfit, and which depth would you choose? Quote train and CV AUC for one depth in each zone.

(b) From the learning curves: how can you tell high bias from high variance? Will adding more training rows fix either one?

(c) From the variance table: how does variance change with depth? What happens to the AUC of the averaged prediction compared with the single trees? Link this to what Random Forest does (Lab 4).

(d) In your own words: what is the bias-variance trade-off, and where do depth 1, depth 3 and depth None sit?

Part 3 — ROC, AUC, Precision-Recall and Thresholds (25 minutes)

Task 3.1 — Compare models with curves

1. Train Logistic Regression (scaled), Decision Tree (max_depth=3) and Random Forest (200 trees, min_samples_leaf=5, random_state=SEED) on the original training data.
2. Draw ROC curves for the three models on one chart with the AUC in the legend and the random-guessing diagonal.
3. Draw Precision-Recall curves for the same models on a second chart. Add a horizontal line at the no-skill level (the stroke rate) and put the average precision in the legend.

Task 3.2 — Choose a decision threshold

1. The hospital wants to catch at least 80% of strokes. Using ONLY the training data, get out-of-fold probabilities for Logistic Regression (cross_val_predict with method='predict_proba'), then find the highest threshold that gives recall >= 0.80 on them.
2. Apply that threshold to the test probabilities. Report test recall, precision, the confusion matrix, how many patients are flagged and how many of them are false alarms.

(a) In Part 1 the recall was very low, yet the ROC AUC of Logistic Regression is much higher than 0.5. How can both be true? What does AUC measure, and what does the 0.5 threshold have to do with it?

(b) Which metric curve is more informative for this data - ROC or Precision-Recall - and why? Use the no-skill line in your answer.

(c) At your chosen threshold, what is the price the hospital pays for the higher recall? Is it acceptable? Argue both sides in 3-4 lines.

Part 4 — Undersampling, Oversampling and SMOTE (40 minutes)

Three common ways to rebalance a training set: RandomUnderSampler (throw away majority rows), RandomOverSampler (copy minority rows) and SMOTE (create new, synthetic minority rows between real minority neighbours). A fourth option needs no resampling: class_weight='balanced'.

Task 4.1 — Compare the methods

1. For Logistic Regression (scaled data) and Random Forest (same settings as Part 3), train with: no resampling, class_weight='balanced', random undersampling, random oversampling and SMOTE. Resample the TRAIN set only; the test set is never touched.
2. Build ONE table (rows = model + method) with: number of training rows, number of strokes in training, Precision, Recall, F1, ROC AUC and PR AUC on the test set.

Task 4.2 — The classic mistake

1. Apply SMOTE to ALL the data first (train + test together), THEN split 80/20, train a Random Forest and evaluate. This is the wrong order.
2. Put its metrics next to the correct SMOTE-after-split result from Task 4.1 in a two-row table.

Hints

- from imblearn.over_sampling import SMOTE, RandomOverSampler; from imblearn.under_sampling import RandomUnderSampler
- X_res, y_res = SMOTE(random_state=SEED).fit_resample(X_train, y_train)
- Do not forget: Logistic Regression uses the scaled data, Random Forest uses the original data.

(a) For Logistic Regression, what changed after resampling and what did NOT change? Compare Recall, Precision and ROC AUC across the methods and explain why.

(b) Did Random Forest react to the methods in the same way as Logistic Regression? Describe one clear difference and give one possible reason.

(c) Compare the wrong-order result with the correct one. Why are the wrong-order numbers so good, and why can they not be trusted? Explain what leaks from where.

(d) If you had to pick one rebalancing approach for this hospital, which would you pick and why? Consider simplicity and the cost of each choice.

Part 5 — Noisy Data (25 minutes)

Real data contains mistakes: wrong labels (a stroke recorded as no stroke) and noisy measurements. You will damage the TRAINING data on purpose and measure the effect on the untouched test set. Compare four models: Logistic Regression, Decision Tree (depth 3), Decision Tree (no depth limit) and Random Forest.

Task 5.1 — Label noise

1. Write a function that flips a given fraction of the training labels at random (0 becomes 1 and 1 becomes 0).

2. For fractions 0, 0.05, 0.10 and 0.20, train each model on the noisy labels and record the test ROC AUC. Repeat 5 times with different random flips and average (use 1 repeat for fraction 0).
3. Show a table (rows = noise fraction, columns = models) and a line plot.

Task 5.2 — Feature noise

1. Add Gaussian noise to the TRAIN values of age, avg_glucose_level and bmi: noise = random normal * (column std) * level. Use levels 0, 0.5, 1 and 2, five repeats each, test set untouched.
2. Show the same kind of table and plot. (If you scale for Logistic Regression, fit the scaler on the noisy training data.)

Task 5.3 — SMOTE and noisy labels

1. Flip 10% of the training labels (3 different random draws). For Logistic Regression and Random Forest compare PR AUC and ROC AUC with and without SMOTE (SMOTE applied after the noise is added).

(a) Which model is most damaged by noise and which is most stable? Quote numbers at the highest noise level and explain using what you learned in Part 2.

(b) Which hurts more here - label noise or feature noise of a comparable level? Give a short reason.

(c) What did SMOTE do to each model when the labels were noisy? Explain why SMOTE can amplify label noise.

Part 6 — Final Decision and Conclusion (15 minutes)

1. Choose your final approach for the hospital (model + rebalancing method or threshold). Build a final table on the test set comparing the default threshold 0.5 with your choice: Precision, Recall, F1, number of false alarms and number of missed strokes.
2. Write an Executive Summary (6-8 lines): what you chose, its recall and precision with numbers, the price paid, how you avoided leakage, and one limitation (think about only 50 stroke patients in the test set, and about the choice of threshold).

Final Reflection

(a) What was the most surprising result in this lab?

(b) If you had one more hour, what would you try next?

Time Allocation

Time	Activity
0-10 min	Part 0 — Setup, cleaning, split
10-25 min	Part 1 — The accuracy trap
25-65 min	Part 2 — Validation curve, learning curves, variance measurement
65-90 min	Part 3 — ROC / PR curves and threshold choice
90-130 min	Part 4 — Undersampling, oversampling, SMOTE and the leakage mistake
130-155 min	Part 5 — Label noise, feature noise, SMOTE with noise
155-170 min	Part 6 — Final decision and Executive Summary
170-180 min	Restart and Run All, save, push to GitHub, submit

Submission Requirements

Submit ONE completed Colab/Jupyter notebook that follows the Part numbers of this manual. At the top of the notebook:

```
# LAB 09 — Bias, Variance and Imbalanced Data
## Student Information
Name:
Student ID:
Section:
GitHub:
Kaggle:
Dataset: Stroke Prediction Dataset (Kaggle)
Dataset Source: https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-
dataset
```

Required

- Every plot and table must be followed by a written interpretation in your own words using YOUR numbers.
- Every question must be answered in a Markdown cell directly under the related code.
- The notebook must run from top to bottom after Restart and Run All. Push the final notebook to your GitHub repository.

Grading Rubric (20 Marks)

Criterion	Marks
Part 0-1 — Correct setup/cleaning/split, accuracy trap shown and explained	3
Part 2 — Validation curve, learning curves, variance measurement and bias-variance explanation	5
Part 3 — ROC/PR curves, threshold chosen from training data only, correct interpretation	4
Part 4 — All resampling methods compared correctly (train only), leakage demonstration explained	5
Part 5 — Label noise, feature noise, SMOTE with noise, interpretation	2
Part 6 — Final choice, Executive Summary, notebook quality and reproducibility	1
Total	**20**

Penalties

- Reporting only accuracy to judge models on this dataset.
- Scaler, imputer, resampler or threshold learned from the test set or from all the data (data leakage): -1 per occurrence (up to -3).
- Plots or tables without a written interpretation, or interpretations that do not match your own numbers.
- Copied notebook (Kaggle or classmate) or hard-coded answers.
- Missing Student Information block or notebook that does not run top to bottom.